

Deep Graph Reinforcement Learning for Solving Multicut Problem

Zhenchen Li¹, Xu Yang¹, *Senior Member, IEEE*, Yanchao Zhang¹, Shaofeng Zeng, Jingbin Yuan¹,
Jiazheng Liu¹, Zhiyong Liu¹, *Senior Member, IEEE*, and Hua Han¹

Abstract—The multicut problem, also known as correlation clustering, is a classic combinatorial optimization problem that aims to optimize graph partitioning given only node (dis)similarities on edges. It serves as an elegant generalization for several graph partitioning problems and has found successful applications in various areas such as data mining and computer vision. However, the multicut problem with an exponentially large number of cycle constraints proves to be NP-hard, and existing solvers either suffer from exponential complexity or often give unsatisfactory solutions due to inflexible heuristics driven by hand-designed mechanisms. In this article, we propose a deep graph reinforcement learning method to solve the multicut problem within a combinatorial decision framework involving sequential edge contractions. The customized subgraph neural network adapts to the dynamically edge-contracted graph environment by extracting bilevel connected features from both contracted and original graphs. Our method can learn to infer feasible multicut solutions end-to-end toward optimization of the multicut objective in a data-driven manner. More specifically, by exploring the decision space adaptively, it implicitly gains heuristic knowledge from topological patterns of instances and thereby generates more targeted heuristics overcoming the short-sightedness inherent in the hand-designed ones. During testing, the learned heuristics iteratively contract graphs to construct high-quality solutions within polynomial time. Extensive

experiments on synthetic and real-world multicut instances show the superiority of our method over existing combinatorial solvers, while also maintaining a certain level of out-of-distribution generalization ability.

Index Terms—Combinatorial optimization, graph neural network, graph partitioning, heuristic algorithm, multicut problem, reinforcement learning.

I. INTRODUCTION

COMBINATORIAL optimization [1] on graphs has always been fundamental to artificial intelligence and pattern analysis. The multicut problem [2], also known as correlation clustering [3], [4], lays the foundation for various graph partitioning problems and attracts considerable attention across areas ranging from data mining [5], [6], [7], [8], computer vision [9], [10], [11], [12] to connectomics [13], [14], [15]. Unlike node classification with supervision [16] or node clustering without connectivity guarantees [17], [18], the multicut problem partitions undirected graphs and discover connected components (i.e., clusters) in an unsupervised manner, based only on node (dis)similarity evidence on edges. Specifically, the multicut problem seeks to find a partitioning that optimizes a total cost or a total gain, hence meeting most of the local evidences. An appealing nature of this problem is that both the number and sizes of clusters can be optimally determined by instances themselves, without prior specification, constraint, or penalization. From this perspective, the multicut problem represents an elegant generalization of several graph partitioning problems, including min-/max-cut [2], [19], $\leq/\geq/k$ -Cut [2], [20], equi-multicut [20], and simple graph partitioning [21]. In addition, it has been extended to many variants, such as lifted multicut [22], asymmetric multiway cut [23], [24], higher-order multicut [25], and higher-order lifted multicut [26].

Unfortunately, like many other combinatorial optimization problems, the multicut problem with an exponentially large number of cycle constraints proves to be NP-hard [3], [4], so exact solutions seem impossible to find within polynomial time. Therefore, instead of exact solvers of exponential complexity, researchers have to resort to approximate solvers with polynomial complexity, most of which are inspired by combinatorial perspectives. In this respect, the solving process of multicut problem can be equivalently formulated as a sequence of elementary transformations of node aggregation [27], [28], [29], [30], edge contraction [22], [31], [32], or node move-making [33], [34], [35], [36], each differing only

Manuscript received 8 August 2023; revised 18 March 2024 and 28 June 2024; accepted 8 August 2024. Date of publication 27 August 2024; date of current version 9 July 2025. This work was supported in part by the Science and Technology Innovation (STI) 2030—Major Projects under Grant 2021ZD0204500 and Grant 2021ZD0204503, in part by the National Natural Science Foundation of China under Grant 32171461 and Grant 61973301, in part by the Youth Innovation Promotion Association of Chinese Academy of Sciences under Grant 2022133, and in part by the Scientific Research Instrument and Equipment Development Project of Chinese Academy of Sciences under Grant YJKYYQ20210022. (Zhenchen Li, Xu Yang, and Yanchao Zhang contributed equally to this work.) (Corresponding author: Hua Han.)

Zhenchen Li, Yanchao Zhang, Jiazheng Liu, and Hua Han are with the Key Laboratory of Brain Cognition and Brain-Inspired Intelligence Technology, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Future Technology, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: lizhenchen2019@ia.ac.cn; zhangyanchao2021@ia.ac.cn; liujiazheng2018@ia.ac.cn; hua.han@ia.ac.cn).

Xu Yang and Zhiyong Liu are with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: xu.yang@ia.ac.cn; zhiyong.liu@ia.ac.cn).

Shaofeng Zeng is with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China (e-mail: shaofeng.zeng@ia.ac.cn).

Jingbin Yuan is with the Key Laboratory of Brain Cognition and Brain-Inspired Intelligence Technology, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: yuanjingbin2020@ia.ac.cn).

Digital Object Identifier 10.1109/TNNLS.2024.3443413

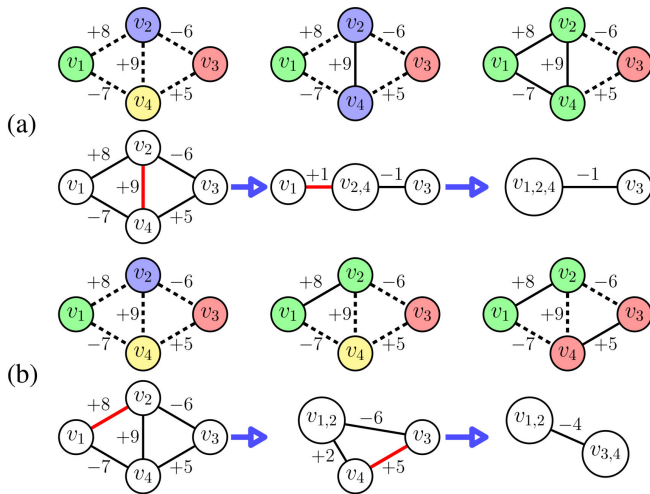


Fig. 1. Illustration of (a) GAEC solver and (b) exact combinatorial solver for solving a minimum cost multicut instance. The costs of edges are marked alongside. In each subfigure, the contracted graphs at the bottom demonstrate sequential edge contractions (highlighted in red) until termination. The top parts of both subfigures illustrate associated partitioning results, in which nodes of the same color form a cluster, and the dashed and solid lines represent edges as cut and uncut, respectively.

in its heuristic strategy. However, all these approximators often give unsatisfactory solutions due to inflexible heuristics driven by hand-designed, conservative mechanisms, which face two major issues discussed below.

First, conventional heuristics fail to systematically exploit the inherent similarities commonly shared by multicut instances arising from the same application domain. Instead, they solve instances case-by-case, with no knowledge being summarized and no experience being generalized. This highlights the need for machine learning to investigate the underlying distribution of instances with similar topological patterns and produce more targeted heuristics in a data-driven manner that could overcome the limitations of insufficient experience and current theory from human beings.

Second, and more importantly, the existing heuristics are almost exclusively greedy in various hand-designed forms [22], [27], [28], [29], [30], [31], [32], [33], [34], thus being suboptimal in most cases. Briefly, they perform transformations step-by-step, each considered most advantageous to the current state regardless of future steps. For example, as shown in Fig. 1(a), the solver of greedy additive edge contraction (GAEC) [22] at each step contracts the edge with a maximum cost on the current contracted graph. However, in order to achieve the optimal partitioning, each decision made as part of the overall partitioning framework should align with its global objective, as indicated in Fig. 1(b). This calls for appropriate machine learning techniques to optimize a decision sequence. In this regard, reinforcement learning [37], [38] with long-term horizons can help to overcome the short-sightedness of conventional greedy heuristics.

Motivated by both issues, we propose a deep graph reinforcement learning method to solve the minimum cost multicut problem (MCMP) within a sequential decision framework. The decision framework is designed by formulating an edge contraction Markov decision process (EC-MDP) over sequential contracted graphs, with a reward function being the

improvement in multicut objectives at a global level. On this basis, we integrate a representation frontend of graph neural networks, named Subgraph Structure2Vec (SS2V), and a decision backend of value-based reinforcement learning, called N -step dueling double deep Q -network (D3QN), into a unified framework. As a result, the entire neural model, SS2V-D3QN, can be learned end-to-end in a data-driven manner toward the well-defined objective. More specifically, the SS2V customized for the EC-MDP environment can help to encode structured states, actions, and Q -functions about non-Euclidean graphs into Euclidean representations, in which bilevel connected features are extracted from both contracted and original graphs. Also, the N -step D3QN aims to learn an optimal parameterized Q -function to derive effective heuristics, with the intention that each executed action of edge contraction gains the most long-term cumulative reward, even if it may be suboptimal for the current individual step. While during testing, the learned models iteratively make decisions for contracting edges and generating solutions within the autoregressive EC-MDP, where each action is uniquely determined by the learned Q -function, conditioned on the current intermediate solution. Thus, solutions to MCMP instances or partitionings of associated original graphs can be obtained by mapping from final contracted graphs.

The contributions of this article are summarized as follows.

- 1) A deep graph reinforcement learning algorithm, SS2V-D3QN, is proposed for sequential edge contractions within the designed EC-MDP decision framework, which aims to learn to solve the NP-hard multicut problem in a data-driven manner. To the best of the authors' knowledge, it is the first to learn to solve the multicut problem end-to-end, with no need for extra postprocessing.
- 2) A novel subgraph neural network, SS2V, is proposed to extract and integrate bilevel connected features within the dynamically edge-contracted graph environment. To the best of the authors' knowledge, the SS2V-D3QN is the first reinforcement learning method for combinatorial decision making in edge contraction or graph partitioning.
- 3) Extensive experiments on synthetic and real-world MCMP instances show that the polynomial SS2V-D3QN significantly outperforms existing combinatorial multicut solvers in solving capability, while also maintaining a certain level of out-of-distribution generalization ability. Our implementation is open-sourced and available at: https://github.com/lizhenchen/DGRL_solve_multicut.

Below, the related work is discussed in Section II, and the problem formulation is defined in Section III. Next, the designed EC-MDP environment is specified in Section IV, and the proposed SS2V-D3QN algorithm is described in Section V. Finally, after the experimental study in Section VI, Section VII summarizes this article and gives some discussion.

II. RELATED WORKS

We exhaustively introduce existing multicut solvers and discuss successful applications of deep graph reinforcement

learning for various combinatorial decision-making tasks including combinatorial optimization problems.

A. Solvers for Multicut Problem

Multicut solvers are typically categorized into three classes: optimization-based, combinatorics-based, and hybrid-based.

1) *Optimization-Based*: Canonical methods for solving the multicut problem rely on (integer [9], [13]) linear programming relaxation [25], [39] within cutting-plane frameworks [40]. Specifically, these methods iteratively tighten an outer relaxation of the multicut polytope [2], [20], [41] until no violated cycle constraints [9], [13] or other separable facet-defining inequalities [25], [39] can be identified. Despite the capability to achieve exactness, they often suffer from high computational complexity, particularly when enumerating constraints is intractable. In practical applications, commercial off-the-shelf solvers like CPLEX [42] are commonly used.

2) *Combinatorics-Based*: Combinatorial heuristics operate directly on graphs and can maintain primal feasible solutions anytime, implicitly sidestepping cycle constraints. Typically, the solving process of multicut problem can be formulated as node aggregation, edge contraction, or node move-making. For node aggregation, algorithms such as BEST [27], FIRST [28], VOTE [29], and PIVOT [30] add nodes to clusters one-by-one [27], [28], [29] or even cluster-by-cluster [30] with various greedy strategies; see [43] for an overview. However, they are heavily sensitive to node ordering. The GAEC solver [22] achieves node aggregation by iteratively contracting the edge with a maximum cost on dynamically contracted graphs, thereby becoming insensitive to the ordering. In addition, the greedy fixation (GF) [31] and the balanced edge contraction (BEC) [32] extend the GAEC by introducing cannot-contract edge relations and cluster-size normalization, respectively. All of the above are in a monotone coarsening fashion and with polynomial complexity, but are prone to falling into poor local minima irreversibly. Reversible move-making heuristics such as Kernighan–Lin with joins (KLjs) [22], [33] and BOEM [34] hence have been proposed for postprocessing, which allow moving nodes between two neighboring clusters or from one cluster to a newly introduced one. Other move-making heuristics iteratively integrate feasible solutions of extra subproblems obtained through combinatorial procedures into monotonely improved solutions, including cut, glue, and cut (CGC) [35] and the fusion move (FM) [36].

3) *Hybrid-Based*: Hybrid solvers [44], [45], [46] combine the perspectives of combinatorics and optimization. Specifically, they reparameterize the edge costs with the help of (Lagrangian) dual optimization and then apply the combinatorial heuristics.

Be aware that the above solvers are all learning-free. Although a recent work [47] takes a step toward solving the multicut problem using machine learning, it can inevitably violate the essential cycle constraints, hence the need for complicated postprocessing involving 0–1 rounding and connected component extraction to return feasible solutions. This is also because it does not follow the equivalent formulation of the solving process but instead applies a penalty method. In contrast, our SS2V–D3QN within the EC-MDP framework

is the first to learn to infer feasible multicut solutions end-to-end in a single stage without extra postprocessing.

B. Deep Graph Reinforcement Learning for Combinatorial Decision-Making

Recent advances in deep learning have led to increasing attention being paid to combinatorial decision-making, especially combinatorial optimization with well-defined objectives.

In 2015, Vinyals et al. [48] proposed a supervised pointer network based on sequence-to-sequence models for solving the traveling salesman problem (TSP). This approach was further improved by reinforcement learning and extended to the Knapsack problem [49]. Most importantly, in 2017, Dai et al. [50] combined graph representation learning and reinforcement learning into a promising scheme called deep graph reinforcement learning for solving several node-selection-based problems incrementally. This scheme was applied or extended to minimum vertex cover [50], max-cut [50], [51], TSP [50], [52], vehicle routing problem [52], graph matching [53], and job-shop scheduling problem [54]. Refer to [55], [56], and [57] for recent surveys of machine learning for combinatorial optimization.

Furthermore, deep graph reinforcement learning has been adopted to explore the decision space of numerous combinatorial decision-making tasks beyond optimization problems. These tasks include molecular graph generation [58], adversarial attacks [59], [60], robustness-oriented graph construction [61], graph exploration [62], key-player discovery [63], and community spatial planning [64]. Refer to [65] for a recent comprehensive review in this respect.

In summary, almost all these deep graph reinforcement learning methods are conducted in a relatively static graph environment (with a constant order of graphs) only involving node/edge selection [50], [51], [52], [53], [62], [63], edge addition [58], [59], [60], [61], or edge deletion [59]. In addition, the method proposed by Zheng et al. [64] is conducted in a dynamic graph environment about node/edge selection. However, a dynamic graph environment (with decreasing orders of graphs) involving edge contraction, required by the multicut problem, remains understudied. To the best of the authors' knowledge, our SS2V–D3QN within the EC-MDP is the first reinforcement learning method for decision-making in edge contraction or graph partitioning, not limited to the multicut problem.

III. PROBLEM FORMULATION

Let $G = (V, E)$ be a simple undirected graph with node set V and edge set E . A cycle of G is a nontrivial path with no repeated nodes except the first and the last one. We denote the set of all cycles of G by $\text{cycles}(G)$. A partitioning of G refers to a partitioning of V into subsets $V_1, \dots, V_k$ satisfying $V_i \neq \emptyset, i = 1, \dots, k, \cup_{i=1}^k V_i = V$, and $V_i \cap V_j = \emptyset, \forall i \neq j$, where k is arbitrary. Note that each subgraph (cluster) $G[V_i], i = 1, \dots, k$ induced by V_i should be connected. As shown in Fig. 2, the multicut M induced by the partitioning $V_1, \dots, V_k$ is a set of edges that straddle different clusters,

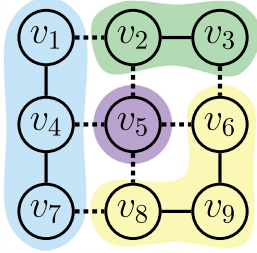


Fig. 2. Partitioning of a graph into four connected subgraphs (clusters) indicated by different colors. The corresponding multicut is composed of the edges (dashed) that straddle different clusters, while the solid lines denote uncut edges.

as follows:

$$M(V_1, \dots, V_k) = \{(u, v) \in E \mid \exists i \neq j : u \in V_i, v \in V_j\}. \quad (1)$$

Such edges are said to be cut. A multicut induced by a partitioning of G is called a multicut of G . We denote by $\mathcal{MC}_G$ the set of all multicuts of G , which relates one-to-one to all partitionings of G . For G , given pairwise node (dis)similarities on edges, a multicut problem aims to find a multicut from $\mathcal{MC}_G$, or a partitioning.

We now introduce a classic version of the multicut problem family, i.e., the MCMP, as follows:

$$\min_{M \in \mathcal{MC}_G} \sum_{e \in M} w_e \quad (2)$$

where the cost function $w: E \mapsto \mathbb{R}$ assigns a cost $w_e \in \mathbb{R}$ to each edge $e \in E$ for separating two endpoints into different clusters. Thus, the MCMP aims to find a partitioning of G that minimizes the total cost of the associated multicut, hence meeting most of the local evidences, by which the endpoints of negative-cost edges can typically be separated, and vice versa. Note that the manner of the cost/gain assignment is the only factor distinguishing the MCMP from other versions of multicut problems such as minimizing disagreements and maximizing agreements [3], [4]. All the versions share an identical perspective for combinatorial decision-making and are NP-hard. Our primary focus is on the MCMP due to its high visibility and the wide interest it attracts.

Equivalently, (2) can be reformulated as a standard 0–1 constrained integer linear programming by deconstructing the solution space $\mathcal{MC}_G$ into Boolean decision variables $y = \{y_e\}_{e \in E}$ for each edge being cut ($=1$) or uncut ($=0$), as follows:

$$\min_{y \in \{0,1\}^{|E|}} \sum_{e \in E} w_e y_e \quad (3a)$$

$$\text{s.t. } \forall C \in \text{cycles}(G) \quad \forall e \in C : y_e \leq \sum_{e' \in C \setminus \{e\}} y_{e'}. \quad (3b)$$

Here, the cycle inequalities (3b) constrain that each cycle is not allowed with exactly one cut edge. Only by these constraints can the induced partitioning be globally valid and consistent. In addition, the convex hull of the feasible solution set is known as the multicut polytope, and the inequalities concerning the cycle C are facet-defining for this polytope if and only if C is chord-less [2].

IV. EC-MDP FRAMEWORK

Following some preliminary knowledge about MDPs and reinforcement learning, the EC-MDP framework customized for solving the multicut problem is introduced.

A. Preliminaries of MDP and Reinforcement Learning

We briefly introduce the MDP defined as $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, which is a sequential decision-making process, where state transitions satisfy the Markov property. In an MDP, a decision-maker called an agent interacts with an environment iteratively. Concretely, the agent at each step observes the current state $s \in \mathcal{S}$ of the environment and takes an action a chosen from the current action space $\mathcal{A}(s)$, receiving a corresponding reward r governed by the reward function $\mathcal{R}(s, a)$. Then, the state of the environment changes to s' , driven by the state transition probability model $\mathcal{P}_{ss'}^a = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$. Continuing in this way, the interaction sequence is collected as a trajectory, $(s_0, a_0, r_{0,1}, s_1, a_1, r_{1,2}, s_2, \dots)$. The agent's goal is to maximize the return—the cumulative discounted reward—on the trajectory, where each immediate reward is discounted over time steps by the factor $\gamma \in (0, 1)$. The agent's behavior is fully determined by a policy $\pi(a|s)$, a probability distribution of possible actions conditioned on states. Given a policy π , the associated state–action value function $Q_\pi(s, a)$ is defined as the expected return of the agent taking the action a at the state s and following the policy π thereafter.

In many cases, the model $\mathcal{P}$ is challenging to know in advance or too complex to estimate, which leads to the development of model-free reinforcement learning agents [37]. These agents can continuously explore environments, record trajectories, and adjust policies through received rewards for maximum expected returns, with no need for explicit models.

B. Edge Contraction MDP (EC-MDP)

Specifying an equivalent sequential decision-making process is a crucial foundation for solving the multicut problem combinatorially. In the following, we provide formal definitions of the edge contraction and the EC-MDP designed for solving the multicut problem.

Let $G = (V, E, W)$ be a weighted undirected graph induced from a given MCMP instance, where the edge weights in $W = \{w_e\}_{e \in E}$ correspond to the edge costs. We represent solving the multicut problem as a sequence of contracted graphs, $\{G'_{(t)} = (V_{(t)}, E_{(t)}, W_{(t)})\}_{t=0}^T$, with $G'_{(0)} := G$. Note that, to distinguish between the nodes on G and those on $G'_{(t)}$, we refer to them in this article by “node” and “vertex,” respectively. Initially, the partitioning consists of one-element clusters, which means that the initial multicut is exactly the edge set E . At each step t , contracting the edge (u, v) in $G'_{(t)}$ means merging the two endpoints u and v as a new vertex v' that is adjacent to any vertex (except u and v themselves) previously adjacent to u or v . Consequently, parallel edges can arise, and they are combined into single ones with their weights summed accordingly. Formally, the newly contracted graph $G'_{(t+1)} = (V_{(t+1)}, E_{(t+1)}, W_{(t+1)})$ is defined by

$$V_{(t+1)} = V_{(t)} \setminus \{u, v\} \cup \{v'\} \quad (4)$$

$$E_{(t+1)} = E_{(t)}(G'_{(t)}[V_{(t)} \setminus \{u, v\}]) \cup \{(v', k) | k \in \mathcal{N}_{t,u} \cup \mathcal{N}_{t,v} \setminus \{u, v\}\} \quad (5)$$

$$w_{t+1,ij} = \begin{cases} w_{t,ij}, & \text{if } i, j \neq v' \\ \sum_{e \in \{(u,j), (v,j)\} \cap E_{(t)}} w_{t,e}, & \text{if } i = v' \\ \sum_{e \in \{(i,u), (i,v)\} \cap E_{(t)}} w_{t,e}, & \text{if } j = v' \end{cases} \quad (6)$$

where $\mathcal{N}_{t,u}$ denotes the neighbor set of u on $G'_{(t)}$. While for the original graph, nodes mapped to u and nodes mapped to v are both newly mapped to the new vertex v' and belong to the same cluster. Thus, the new partitioning is composed of the current clusters, and the multicut solution is also updated by definition. Note that the properties of edge contraction (4) and (5) ensure that the contracted graphs correspond one-to-one with partitioning results. Furthermore, the update criterion for edge weights (6) guarantees that the dynamically contracted graphs can always maintain the objective function of original MCMPs. This is because the objective value of a multicut solution is equal to the total edge weight on the associated contracted graph. Be aware that the maximum number of times the original graph can be contracted is $|V|$ minus the number of connected components, due to (4).

Based on the above, the EC-MDP, a fully observable graph environment about edge contraction, can be formally defined as follows.

- 1) *State Space*: A state s_t of the environment at time t can be observed from both the current contracted graph $G'_{(t)}$ and the original graph G with the current cut marks indicating for each edge whether it is cut. The state space $\mathcal{S}$ is non-Euclidean and exponentially large.
- 2) *Action Space*: An action at contraction step t , $a_{t,e}$, is to contract an edge e in $G'_{(t)}$. The discrete action space $\mathcal{A}(s_t)$ corresponds to the edge set $E_{(t)}$ of $G'_{(t)}$.
- 3) *Transition Model*: At step t , with the current state s_t and the action edge e to be contracted, the state transition model $\mathcal{P}$ is deterministic (i.e., one-hot) for s_{t+1} involving both the newly contracted graph defined in (4)–(6) and the original graph with new cut marks. However, $\mathcal{P}$ is too complex to represent formally, due to the non-Euclidean and exponentially large state space.
- 4) *Reward Shaping*: The reward function is shaped as the improvement in the multicut objective by transition, as $\mathcal{R}(s_t, a_{t,e}) = \text{obj}(s_t) - \text{obj}(s_{t+1})$. Here, $\text{obj}(s_t)$ is an objective value of the multicut solution induced by state s_t , namely, a total edge weight on the current contracted graph $G'_{(t)}$, $\sum_{e \in E_{(t)}} w_{t,e}$. Such a stepwise intermediate reward is designed to guide the agent's behavior toward optimization of the global multicut objective. Equivalently, the reward can also be quantified as the weight of the action edge e on $G'_{(t)}$, $w_{t,e}$.
- 5) *Discount Factor*: A discount factor $\gamma \in (0, 1)$ motivates the agent to seek more rewards within a finite time horizon. The choice of γ can affect the tradeoff between short-term and long-term planning. Specifically, a lower value prioritizes short-term gains, typically contracting the edge with a maximum weight, while a higher value

favors long-term planning despite experiencing more uncertainties. An intermediate value can balance the two.

- 6) *Termination*: When there is no edge with a positive weight on the current contracted graph, the EC-MDP terminates.

With the EC-MDP designed, we anticipate that the agent behaves with an optimal policy. In fact, once the optimal Q -function $Q^*(s, a)$ is obtained, an optimal policy π^* can be naturally derived as a deterministic greedy policy [66]

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \arg \max_{a' \in \mathcal{A}(s)} Q^*(s, a') \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

Following this policy, the agent can iteratively construct an optimal multicut solution through a sequence of edge contractions until the termination condition is met. However, the theoretically optimal Q -function is too complex to find and represent. Below, we will demonstrate how approximate ones can be found.

V. SS2V–D3QN

In this section, we introduce the proposed deep graph reinforcement learning method, SS2V–D3QN, for solving the multicut problem within the EC-MDP decision framework, as illustrated in Fig. 3. By exploring the decision space adaptively, the method implicitly gains heuristic knowledge from the topological patterns of multicut instances, in order to find optimal heuristics for sequential edge contractions. Specifically, the SS2V network helps to provide the D3QN algorithm with representations of states and actions on the dynamically edge-contracted graph environment, and the D3QN with long-term horizons explores the decision space to find optimal edge-contraction policies toward the global multicut objective.

To elaborate, since the model $\mathcal{P}$ is too complex to represent formally, we employ model-free reinforcement learning techniques to explore the EC-MDP environment adaptively. Given the discrete action space, we choose to utilize a value-based reinforcement learning method. However, the non-Euclidean and exponentially large state space still poses a challenge in representing the state–action value function due to the curse of dimensionality. Hence, the idea of function approximation is adopted, and $Q(s, a)$ is parameterized as $\hat{Q}(s, a; \theta)$, where θ denotes parameters. Then, deep reinforcement learning algorithms like DQN [67] can be used to learn the optimal parameterized Q -function and generate effective policies efficiently. Nevertheless, a key challenge we face is how to encode the states and actions on the dynamically edge-contracted graph environment with structural awareness.

To address this challenge, we propose the SS2V customized to characterize structural features on the EC-MDP graph environment, incorporating graph topology, edge weights, and cut marks from contracted and original graphs. On this basis, the Q -function is enabled to be parameterized. Then, we apply the N -step D3QN training framework [68], [69] to fit the optimal parameterized Q -function and generate effective policies. Finally, to obtain high-quality multicut solutions during testing, we employ ensemble inference as a test-time augmentation technique.

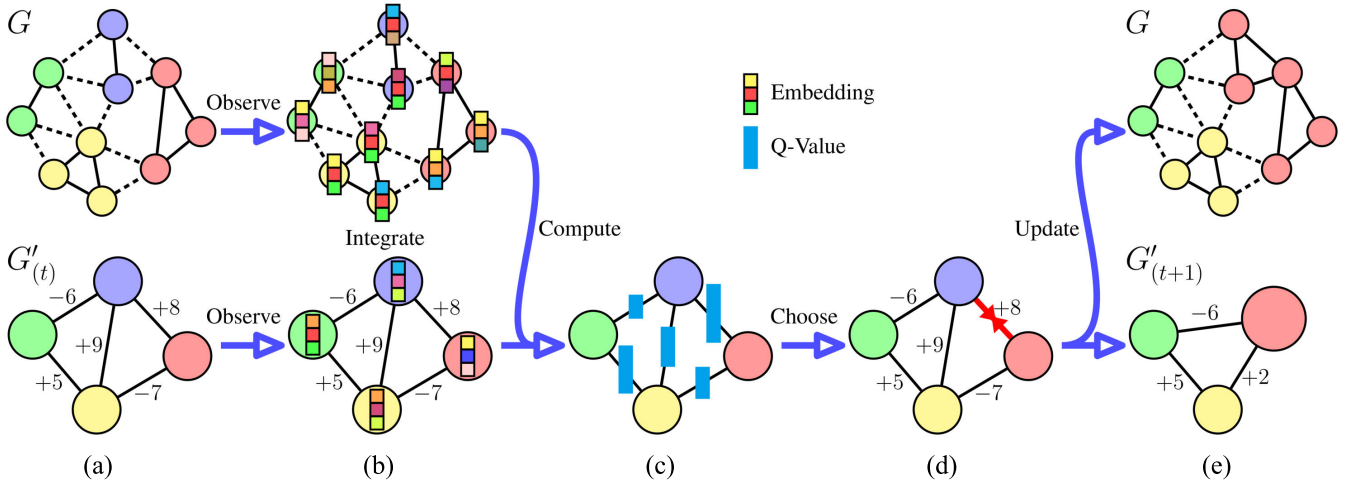


Fig. 3. Illustration of the proposed method for inferring multicut solutions within the EC-MDP environment at contraction step t . At time t , the current state involving both the current contracted graph $G'_{(t)}$ and the original graph G with current cut marks is observed by the SS2V network to capture and integrate bilevel connected structural features into embeddings. Then, the Q values for the edges in the current contracted graph can be computed through parameterization, and the edge (in red) with a maximum Q value is chosen for contraction as the action. Consequently, the newly contracted graph at time $t+1$, $G'_{(t+1)}$, is obtained by definition (4)–(6), and partitioning of the original graph can also be updated. (a) State at time t . (b) SS2V. (c) Q -values. (d) Action. (e) State at time $t+1$.

A. Subgraph Structure2Vec (SS2V)

The state space of the EC-MDP graph environment is non-Euclidean, and the number of states grows exponentially in the order of the graph. Hence, there is a pressing need for Euclidean representations of non-Euclidean graphs in a topology-aware manner, which would enable reinforcement learning agents to discriminate among actions over any state at any time in a goal-directed fashion. To this end, graph neural networks [70], [71], [72], [73] are ideal tools, which are considered flexible and expressive. In this section, we propose the SS2V, a graph embedding network inspired by the mean-field inference (i.e., a theoretically well-grounded graphical model inference algorithm) [74] and the vanilla Structure2Vec [50] but tailored for the dynamically edge-contracted graph environment rather than the statically node-selected one [50]. The SS2V network can extract and integrate bilevel connected features from both contracted and original graphs.

For the EC-MDP, the state at time t , s_t , involves two graphs: the current contracted graph $G'_{(t)}$ and the original graph G with the current cut marks $\{\phi_{t,e}\}_{e \in E}$. The relationship between the two graphs is such that each vertex u' on the coarse-grained contracted graph corresponds to a subgraph (cluster) $G[V_{u'}]$ on the fine-grained original graph, where $V_{u'}$ denotes the set of nodes mapped to u' . Note that these nodes of the set have been aggregated together through previous contractions. Below, we develop two subnetworks; one for each graph.

For the current contracted graph $G'_{(t)}$, we aim to embed each vertex u' as a p' -dimensional vector, $\mu_{u'}$, which represents its local topologies with weight information and is initialized as a zero vector, i.e., $\mu_{u'}^0 = \mathbf{0}$. Based on the weights of incident edges, we derive static local features for each vertex, using neural networks, nonlinear activation, and pooling operation, as follows:

$$\xi_{u'} = \sum_{v' \in \mathcal{N}_{t,u'}} \sigma(\theta_1 w_{u'v'}) \quad (8)$$

where $\theta_1 \in \mathbb{R}^{p'}$ denotes parameters of a linear fully connected network, $\sigma(\cdot)$ indicates the well-established rectified linear unit (ReLU), and the summation over neighbors ensures permutation-invariant message aggregation. Incorporating these static local features, the embedding of each vertex can then be updated iteratively by message passing from its adjacent vertices and itself, as follows:

$$\mu_{u'}^k = \sigma \left(\theta_2 \xi_{u'} + \theta_3 \mu_{u'}^{k-1} + \theta_4 \sum_{v' \in \mathcal{N}_{t,u'}} \mu_{v'}^{k-1} \right), \quad 1 \leq k \leq K' \quad (9)$$

where θ_2, θ_3 , and $\theta_4 \in \mathbb{R}^{p' \times p'}$ are shared across all vertices and all iterations. After K' rounds of message passing, the permutation-invariant embedding of vertex $\mu_{u'}$ can be read out as $\mu_{u'} = \mu_{u'}^{K'}$, which describes its K' -hop neighborhood.

Similarly, we encode each node u on the original graph G as a p -dimensional vector, μ_u , via

$$\xi_u = \sum_{v \in \mathcal{N}_{0,u}} \sigma(\theta_5 [w_{uv}, \phi_{t,uv}]) \quad (10)$$

$$\mu_u^k = \sigma \left(\theta_6 \xi_u + \theta_7 \mu_u^{k-1} + \theta_8 \sum_{v \in \mathcal{N}_{0,u}} \mu_v^{k-1} \right), \quad 1 \leq k \leq K \quad (11)$$

where $\theta_5 \in \mathbb{R}^{p \times 2}$, $\theta_6, \theta_7, \theta_8 \in \mathbb{R}^{p \times p}$, the current cut mark $\phi_{t,uv}$ indicates the edge (u, v) being cut ($=0$) or uncut ($=1$) at time t , and $[\cdot, \cdot]$ represents concatenation. Also, the initial node embeddings are zero vectors and the final ones are obtained with K -round message passing, i.e., $\mu_u = \mu_u^K$. Be aware that the major difference between the two subnetworks is whether the cut marks are involved.

To obtain a more comprehensive embedding for each vertex u' related to the current action space, we concatenate the bilevel connected features involving u' from both the current

contracted graph and the original graph, followed by a linear network, as follows:

$$h(u') = \theta_9 \left[\mu_{u'}, \sum_{u \in V_{u'}} \mu_u, |V_{u'}| \right] \quad (12)$$

where $\theta_9 \in \mathbb{R}^{q \times (p+p'+1)}$ denotes parameters. The embedding $h(u')$ of vertex u' captures both its external and internal structural features at the current state, complementing to each other. With regard to the internal descriptors, the summation over subgraph $G[V_{u'}]$, along with the accompanying order, is not only feature integrated but also permutation invariant.

B. Q -Function Parameterization

On the basis of the above, we aim to further parameterize $Q(s, a)$ as $\hat{Q}(s, a; \theta)$ by vectorizing the states and actions.

The vectorization for state s_t can be naturally obtained through global integration and ReLU activation, as follows:

$$h(s_t) = \sigma \left(\left[\sum_{u' \in V_{(t)}} h(u'), |V_{(t)}| \right] \right). \quad (13)$$

Here, the summation across the entire action graph (i.e., the current contracted graph), accompanied by an explicit order, could distinguish between early and later states and has the potential to generalize to larger instances. In addition, the descriptor for the action edge (u', v') can be obtained by taking both the two corresponding vertices and its own weight into account, as follows:

$$h(a_{t,u'v'}) = \sigma \left([\theta_{10}(h(u') + h(v')), \theta_{11}w_{t,u'v'}] \right) \quad (14)$$

where $\theta_{10} \in \mathbb{R}^{q \times q}$ and $\theta_{11} \in \mathbb{R}^{q'}$.

Now, we can represent the parameterized Q -function. To better distinguish the effects of states and actions on Q -values, we disassemble $\hat{Q}(s_t, a_{t,e}; \theta)$ into the state value function $\hat{V}(s_t; \theta)$ and the advantage function $\hat{A}(s_t, a_{t,e}; \theta)$ [68], as follows:

$$\hat{Q}(s_t, a_{t,e}; \theta) = \hat{V}(s_t; \theta) + \hat{A}(s_t, a_{t,e}; \theta) - \frac{1}{|E_{(t)}|} \sum_{e' \in E_{(t)}} \hat{A}(s_t, a_{t,e'}; \theta) \quad (15)$$

where $\hat{V}(s_t; \theta)$ is made equal to the mean value of $\hat{Q}(s_t, a_{t,e}; \theta)$ over the current action space in this dueling architecture. Both the functions can be modeled as two-layer perceptrons

$$\hat{V}(s_t; \theta) = \theta_{13}^T (\sigma(\theta_{12} h(s_t))) \quad (16)$$

$$\hat{A}(s_t, a_{t,e}; \theta) = \theta_{15}^T (\sigma(\theta_{14} [h(s_t), h(a_{t,e})])) \quad (17)$$

where $\theta_{12} \in \mathbb{R}^{b \times (q+1)}$, $\theta_{13} \in \mathbb{R}^b$, $\theta_{14} \in \mathbb{R}^{d \times (2q+q'+1)}$, and $\theta_{15} \in \mathbb{R}^d$.

Note that $\theta = \{\theta_i\}_{i=1}^{15}$ constitutes all the parameters to be learned, which are shared across any state, regardless of the contraction step t . Also note that the linear layers associated with $\{\theta_i\}_{i=9}^{15}$ also incorporate learnable bias parameters, which have been omitted from the current presentation for conciseness.

C. N -Step D3QN Training Framework

The N -step D3QN framework is applied for training.

In Q -learning [66], the optimal Q -function suggests an optimal policy. To optimize the parameterized Q -function, $\hat{Q}(s, a; \theta)$, we employ the D3QN framework, which extends the vanilla DQN [67] with the dueling architecture [68] as described in Section V-B and the double Q -learning [69]. The latter can reduce the over-estimation issue by untangling action selection and action evaluation. These improvements facilitate accurate evaluation and efficient convergence. In addition, the collection of N -step rewards, defined as $r_{t,t+N} = \sum_{k=0}^{N-1} \gamma^k \mathcal{R}(s_{t+k}, a_{t+k})$, offers more comprehensive estimates of long-term cumulative rewards compared with one-step ones, enabling faster learning. With the experience replay memory $\mathcal{M}$, the loss function of the framework is chosen as an expected mean squared error

$$\mathcal{L}(\theta; \mathcal{M}) = \mathcal{E}_{(s_t, a_t, r_{t,t+N}, s_{t+N}) \sim \mathcal{M}} (\hat{Q}(s_t, a_t; \theta) - x)^2 \quad (18)$$

$$x = \begin{cases} r_{t,t+N}, & \text{if end at time } t+N \\ r_{t,t+N} + \gamma^N \hat{Q}(s_{t+N}, \hat{a}; \theta^-), & \text{otherwise} \end{cases} \quad (19)$$

$$\hat{a} = \arg \max_{a' \in \mathcal{A}(s_{t+N})} \hat{Q}(s_{t+N}, a'; \theta) \quad (20)$$

where θ^- denotes the parameters of a target network not involved in back-propagation.

The entire framework is shown in Algorithm 1.

Algorithm 1 N -Step D3QN Training Framework

Input: Graph (instance) distribution $\mathcal{D}$

Output: θ

Parameters: episode number J , exploration rate function $\varepsilon(\cdot)$, minimum memory size $C_{\min}$, updating frequency f_1 , learning frequency f_2

Initialize θ randomly

Initialize $\mathcal{M}$ of capacity C as null

```

1: for episode  $j = 1$  to  $J$  do
2:   if  $(j - 1) \bmod f_1 == 0$  then
3:      $\theta^- = \theta$ 
4:   end if
5:   Sample graph batch from  $\mathcal{D}$ 
6:   for step  $t = 0, 1, 2, \dots$  do
7:     for each graph do
8:        $e = \begin{cases} \text{random edge from } E_{(t)}, & \text{with prob. } \varepsilon(j), \\ \arg \max_{e' \in E_{(t)}} \hat{Q}(s_t, a_{t,e'}; \theta), & \text{otherwise} \end{cases}$ 
9:       Take action  $a_{t,e}$  and generate  $G'_{(t+1)}$  by (4)(5)(6)
10:      if  $t \geq N - 1$  then
11:        Add  $(s_{t-N+1}, a_{t-N+1}, r_{t-N+1,t+1}, s_{t+1})$  to  $\mathcal{M}$ 
12:      end if
13:    end for
14:    if  $|\mathcal{M}| \geq C_{\min}$  and  $t \bmod f_2 == 0$  then
15:      Sample tuple batch  $B \sim \mathcal{M}$ 
16:      Update  $\theta$  by optimizer over (18) for  $B$ 
17:    end if
18:  end for
19: end for

```

D. Solution Generation With Ensemble Inference

Once the parameters θ have been learned, we can generate a solution to each MCMP instance in an autoregressive manner [56]. Specifically, the associated original graph is iteratively contracted by an agent according to a deterministic policy induced by $\hat{Q}(s, a; \theta)$, until the termination condition is met. Here, the policy with long-term horizons dictates that an action at each step is executed on the edge with a maximum Q value instead of a maximum cost, referring to (7). A partitioning or a multicut solution can then be obtained by mapping from the final contracted graph, where each vertex indicates a cluster. The computational complexity is considered to be $O(|V||E|)$, because even a connected original graph can be contracted at most $|V| - 1$ times, and each contraction is based on message passing via edges and Q value calculations for action edges.

To enhance solving capability during testing, we utilize ensemble inference as a test-time augmentation technique. Specifically, $H \in \mathbb{Z}^+$ policies of high quality making up an agent team is employed to infer independently on each instance, and the highest-quality result obtained is chosen, measured by a total cost on the final contracted graph, or a cumulative undiscounted reward. These H policies perform best on a validation set, where each policy is saved at the end of an episode. Each policy in its own style is considered to behave well on specific subdistribution instances, and their integration offers complementary advantages to handle almost any instance from the given distribution. In short, our approach integrates multiple independent strong policies into a single stronger one, inspired by ensemble learning.

VI. EXPERIMENTAL STUDY

In this section, the proposed SS2V-D3QN algorithm is evaluated against almost all existing combinatorial multicut solvers on both synthetic and real-world MCMP instances of various scales.

A. Baseline Solvers

To the best of our ability, we take almost all the combinatorial multicut solvers for comparisons, including BEST [27], FIRST [28], VOTE [29], PIVOT [30], GAEC [22], GF [31], BEC [32], CGC [35], FM [36], and KLj [22], [33], as introduced in Section II-A. The first seven are in a monotone coarsening fashion, while the last three allow reversible move-making. Among them, node aggregation-based solvers [27], [28], [29], [30] are with computational complexity of $O(|V|^2)$, and edge contraction-based ones [22], [31], [32] are also polynomial. For the others, the GAEC is utilized for warm starts. In addition, for the FM, Randomized Watershed [36] with 5% node seeds and a smoothing factor of 2.0 is applied as proposal generators, and the GAEC and KLj are employed as internal solvers, respectively, with stopping criteria of the maximum iteration number of 1000 or 25 consecutive iterations without improvements. As for the CGC and the KLj, they stop until no more improvement is possible. In addition, the commercial exact solver, CPLEX, is adopted as a golden standard, which, together with GAEC, CGC, FM, and KLj,

are implemented by nifty,¹ a publicly available Python library with a C++ backend. The other baselines are implemented in Python on our own.

B. Experimental Settings

All SS2V-D3QN models are trained, validated, and tested on Linux servers with Python 3.6.9, PyTorch 1.7.0, and Intel Core i9-10900X CPUs at 3.70 GHz.

The training, validation, and test sets contain 500, 50, and 50 MCMP instances (undirected weighted graphs), respectively.

For the EC-MDP, γ is set to 0.95. For the ensemble inference, H can be set to 10. With regard to the SS2V network and associated Q -parameterization, $K = K' = 2$, $p = p' = 64$, $q = q' = 64$, $b = 16$, and $d = 32$. As for the three-step D3QN, the graph batch size is 10, the exploration rate ε linearly decreases from 0.3 to 0.05 across all episodes, and the parameters are optimized by AdamW [75] with cyclical learning rates [76]. Within the initial cycle of 500 episodes, the learning rate increases from 0.001 to 0.005 and then decreases back to 0.001, the boundary values of which undergo an exponential decay of 0.98 over cycles. In addition, f_1 is 5. Other hyperparameters for small, medium, and large-scale synthetic instances are discussed separately. The episode numbers J are 5×10^4 , 2×10^4 , and 2×10^4 , respectively. The memory capacities C are 1.5×10^5 , 1.5×10^5 , and 2.0×10^5 , respectively, with the minimum sizes $C_{\min}$ set at one quarter of their respective values. The tuple batch sizes are 64, 96, and 128, respectively. The learning frequencies f_2 are 5, 10, and 50, respectively. For real-world instances, refer to the case of medium-scale synthetic ones.

C. On Synthetic Instances

We evaluate the performance on synthetic instances of modest scales, generated by the following random graph models. For each model, small, medium, and large-scale graphs are considered, with 20, 40, and 60 nodes, respectively. In addition, edge weights are sampled from a continuous uniform distribution over the interval $[-1, 1]$, denoted by $U(-1, 1)$.

- 1) *Barabási-Albert (BA)* [77]: Under this model, a graph is grown by attaching new nodes, each with four edges that are preferentially attached to existing nodes with a high degree.
- 2) *Watts-Strogatz (WS)* [78]: Initially, each node is joined with its six-nearest neighbors in a ring. Then, the probability of rewiring each edge is set at 0.2.
- 3) *Erdős-Rényi (ER)* [79]: The ER model satisfying connectedness chooses each of the possible edges (node pairs) with a predefined probability. Note that the number of edges grows quadratically with the number of nodes conditioned on this probability, whereas the previous two models exhibit linear growth. Hence, the probabilities for the small, medium, and large graphs are decreasingly set to 0.175, 0.150, and 0.125, respectively. If larger ER graphs are required, this value is reduced to 0.1.

¹<https://github.com/DerThorsten/nifty/>

TABLE I
PERFORMANCE COMPARISON FOR SS2V-D3QN WITH BASELINE SOLVERS ON MCMP INSTANCES GENERATED FROM THREE RANDOM MODELS OF THREE DIFFERENT ORDERS. IN EACH ITEM, A TOTAL OBJECTIVE VALUE OF ALL OBTAINED SOLUTIONS FOR THE TEST INSTANCE SET IS REPORTED (LOWER IS BETTER)

Solvers	BA			WS			ER		
	$ V =20$	$ V =40$	$ V =60$	$ V =20$	$ V =40$	$ V =60$	$ V =20$	$ V =40$	$ V =60$
CPLEX	-581.90	-1265.01	-1908.54	-557.81	-1144.28	-1745.44	-380.84	-1137.81	-1985.27
CGC	-573.37	-1226.85	-1838.66	-544.23	-1128.68	-1721.31	-378.35	-1111.37	-1931.48
FM (GAEC)	-574.82	-1224.41	-1836.23	-547.84	-1129.43	-1719.79	-378.43	-1112.33	-1929.74
FM (KLj)	-575.17	-1224.41	-1836.40	-547.84	-1129.56	-1720.54	-378.43	-1112.78	-1931.22
KLj	-575.59	-1239.39	-1851.10	-549.83	-1129.79	-1723.29	-378.42	-1114.07	-1944.66
BEST	-401.27	-857.14	-1186.25	-362.04	-732.10	-1193.23	-308.01	-810.62	-1350.20
FIRST	-301.99	-549.12	-786.19	-277.18	-539.26	-898.77	-258.59	-627.49	-1009.66
VOTE	-487.16	-1068.83	-1615.97	-467.34	-967.42	-1479.99	-325.45	-893.95	-1495.63
PIVOT	-290.23	-580.12	-813.33	-280.42	-557.92	-840.45	-223.37	-555.43	-867.79
GAEC	-569.25	-1215.11	-1821.72	-541.43	-1117.45	-1711.37	-378.12	-1099.01	-1908.93
GF	-527.13	-1133.52	-1699.07	-523.10	-1063.56	-1646.09	-373.03	-1065.51	-1775.08
BEC	-565.85	-1212.12	-1828.83	-539.02	-1110.22	-1708.83	-377.27	-1106.75	-1893.04
Ours ($H=1$)	<u>-570.07</u>	<u>-1221.21</u>	-1824.44	<u>-547.03</u>	<u>-1119.21</u>	<u>-1715.03</u>	<u>-378.51</u>	<u>-1111.38</u>	<u>-1915.93</u>
Ours ($H=10$)	-576.13	-1250.93	-1855.71	-552.11	-1130.25	-1723.38	-378.81	-1119.63	-1945.57

For each graph model of each order, the top ranking among all solvers except CPLEX is highlighted in **bold**, while the best one among the solvers in a monotone coarsening fashion is underlined.

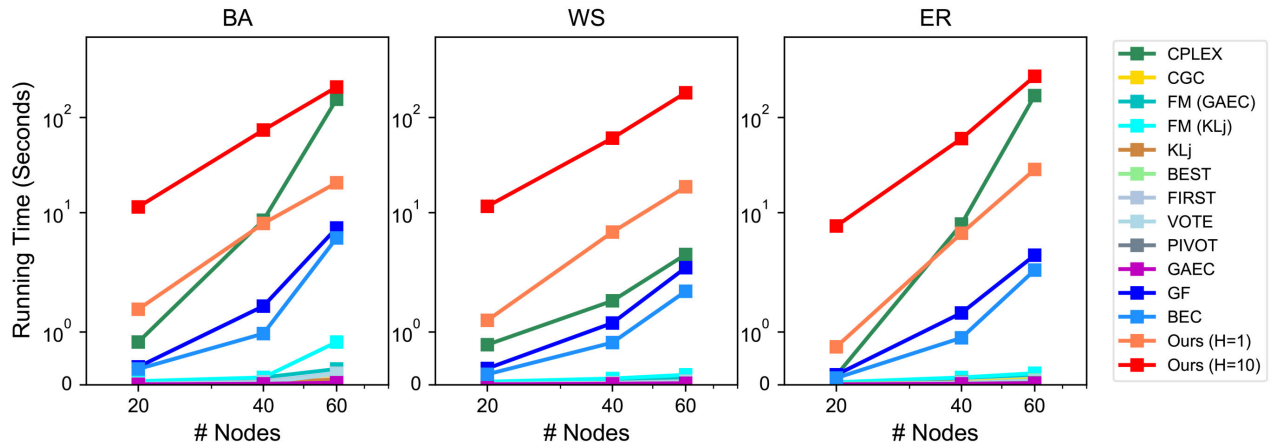


Fig. 4. Comparison of running time for SS2V-D3QN with baseline solvers on MCMP instances from three random graph models of three different orders. For each test set, a total time expended by each solver in solving all 50 instances is recorded. Note that all the axes employ logarithmic scales.

As presented in Table I, the proposed SS2V-D3QN shows its impressive capability to learn to infer feasible multicut solutions toward optimization of the multicut objective in a data-driven manner. There are two noteworthy observations to highlight. First, combined with the ensemble inference ($H=10$), the learning-based SS2V-D3QN significantly outperforms all the learning-free combinatorial solvers on MCMP instances generated from various models of random graphs of various orders. Second, without the ensemble inference ($H=1$), it is still superior among the combinatorial solvers of monotone coarsening in most cases.

As demonstrated in Fig. 4, our learning-based method, SS2V-D3QN ($H=1$), while not considered as fast as learning-free heuristics due to the computational overhead involved in computing edge candidates for contraction, always takes less than half a second on average to handle an instance involving up to 60 nodes, which can be deemed acceptable. Furthermore, its polynomial complexity has been

experimentally corroborated, because the running time is approximately linear w.r.t. the orders of graphs in the log-log coordinate systems. In addition, additional acceleration in deployment could be achieved in future work by techniques such as model compression, model deployment tools, and hardware acceleration.

As for scalability, the reasons for not training on much larger MCMP instances are as follows. First, much larger instances involve more complex topological patterns, demanding a much higher model capacity of the SS2V network to capture. Second, and more importantly, much larger instances entail a more involved decision-making environment and more edge-contraction steps, which makes the reinforcement learning algorithm, D3QN, more difficult to explore and exploit. In Section VI-D, we will evaluate the generalization ability of the collected SS2V-D3QN models trained on the modestly-scaled instances to much larger ones from the same graph model.

TABLE II
EVALUATION FOR OUT-OF-DISTRIBUTION GENERALIZATION ABILITY ACROSS DIFFERENT MODELS OF THE SAME ORDER AND ACROSS DIFFERENT ORDERS OF THE SAME MODEL. FOR INTERSECTION CELLS OF SOURCE DOMAINS (ROWS) AND TARGET DOMAINS (COLUMNS), RESULTS WITHOUT/WITH ENSEMBLE INFERENCE ($H = 1$ AND 10) ARE REPORTED

General. Source	Target								
	BA-20	BA-40	BA-60	WS-20	WS-40	WS-60	ER-20	ER-40	ER-60
BA-20	-570.1 /-576.1	-1226.6 /-1251.6	-1827.6/-1873.1	-545.2/-550.2	—	—	-377.9/-379.2	—	—
BA-40	-567.1/-576.0	-1221.2/-1250.9	-1835.6 /-1878.0	—	-1122.3/-1132.1	—	—	-1106.9/- 1123.5	—
BA-60	-569.3/-572.3	-1221.6/-1235.9	-1824.4/-1855.7	—	—	-1712.5/-1723.6	—	—	-1907.8/-1936.5
WS-20	-569.5/- 576.3	—	—	-547.0 /-552.1	-1118.1/- 1135.2	-1710.6/- 1732.4	-377.8/-379.4	—	—
WS-40	—	-1225.3/-1242.7	—	-543.0/-547.3	-1119.2/-1130.3	-1713.8/-1725.6	—	-1106.3/-1116.5	—
WS-60	—	—	-1825.6/-1858.0	-545.4/-548.5	-1122.9/-1130.1	-1715.0 /-1723.4	—	—	-1910.3/-1937.4
ER-20	-570.0/-574.4	—	—	-544.0/-549.0	—	—	-378.5/-378.8	-1106.9/-1119.5	-1899.0/-1936.4
ER-40	—	-1221.2/-1246.4	—	—	-1124.9 /-1130.1	—	-378.6 /-378.6	-1111.4 /-1119.6	-1915.9 /-1943.8
ER-60	—	—	-1826.6/-1865.0	—	—	-1711.9/-1726.5	-377.7/- 379.5	-1103.8/-1119.5	-1915.9 /- 1945.6

For each instance set of target domain, the best results without and with ensemble inference are both highlighted in **bold**.

TABLE III
EVALUATION FOR GENERALIZATION ABILITY TO MUCH LARGER INSTANCES FROM THE SAME GRAPH MODEL. IN EACH ITEM, A TOTAL OBJECTIVE VALUE OF ALL OBTAINED SOLUTIONS FOR THE TEST INSTANCE SET IS REPORTED (LOWER IS BETTER)

Solvers	BA			WS			ER		
	$ V =80$	$ V =100$	$ V =120$	$ V =80$	$ V =100$	$ V =120$	$ V =80$	$ V =100$	$ V =120$
CPLEX	-2709.08	-3446.75	-4048.18	-2314.00	-2904.93	-3532.39	-2834.69	None	None
BEST	-1760.76	-2202.96	-2617.41	-1543.63	-1927.18	-2364.24	-1968.59	-2528.41	-3308.52
FIRST	-1225.05	-1405.74	-1829.77	-1122.17	-1473.44	-1800.48	-1371.58	-1719.81	-2139.03
VOTE	-2308.30	-2932.12	-3435.55	-1933.28	-2410.69	-2938.93	-2123.44	-2853.76	-3787.29
PIVOT	-1269.07	-1585.62	-1768.56	-1091.15	-1345.89	-1674.71	-1251.18	-1608.98	-1945.44
GAEC	-2595.64	-3303.58	-3856.19	-2273.25	-2851.38	-3475.20	-2706.26	-3762.88	-4930.01
GF	-2406.76	-3069.93	-3589.43	-2188.67	-2735.43	-3359.89	-2535.65	-3383.56	-4311.42
BEC	-2584.12	-3289.28	-3838.56	-2260.76	-2841.82	-3467.58	-2688.82	-3696.00	-4803.53
CGC	-2620.14	-3334.82	-3895.40	-2278.80	-2864.50	-3484.21	-2729.50	-3798.97	-4980.45
FM (KLj)	-2644.74	-3352.29	-3940.37	-2287.82	-2875.97	-3504.88	-2760.83	-3871.35	-5124.08
KLj	-2636.81	-3348.92	-3916.07	-2285.67	-2871.81	-3490.49	-2763.20	<u>-3852.47</u>	<u>-5108.66</u>
Ours ($H=10$)	-2651.18	-3356.45	<u>-3928.96</u>	-2293.56	<u>-2875.66</u>	<u>-3498.54</u>	<u>-2761.95</u>	-3841.72	-5025.79

'None' indicates unattainable solutions within one week, while other values are obtained within three hours.

For each larger model, the top ranking among all solvers except CPLEX is highlighted in **bold**, while the second place is underlined.

TABLE IV
ABLATION STUDY FOR ITERATION NUMBERS OF SS2V NETWORK ON BA TEST INSTANCE SET WITH THE ORDER 40. "NONE" INDICATES THE ABSENCE OF A CORRESPONDING SUBNETWORK, THEREBY UTILIZING DOUBLE THE CHANNELS TO EXTRACT FEATURES ON ANOTHER GRAPH

SS2V K	K'			
	None	2	3	4
None	—	-1241.24	-1247.48	-1246.37
2	-1242.46	-1250.93	-1248.45	-1249.55
3	-1240.22	-1248.47	-1248.14	-1239.23
4	-1234.51	-1247.70	-1247.96	-1243.80

The top ranking among all the settings is highlighted in **bold**.

D. Out-of-Distribution Generalization

We evaluate the out-of-distribution generalization ability to unseen instances across different graph models of the same order and across different orders of the same model by transferring from source domains to target domains. As shown in Table II, this generalization ability is roughly confirmed due possibly to interdistribution similarity. More specifically,

the majority of the top-ranking results cluster around the diagonal, indicating stronger generalization ability within the same model compared with across different models of the same order. Nonetheless, there are still a few cases of exceptional generalization across different models. This can be attributed to the greater diversity of collected policies for different random graph models than for their own model, which helps to take full advantage of ensemble inference. Moreover, when considering different orders within the same model, policies learned on smaller instances sometimes exhibit impressive generalization performance on larger ones, particularly for the scale-free model BA with preferential attachment. This may be because policies trained on smaller orders can experience more updates during training (refer to J and f_2), and also due to the structure insensitivity of scale-free models to varying orders.

Beyond the above instances of modest scales, the ability to generalize to much larger instances from the same graph model is of great interest, given its applicable value. The SS2V-D3QN method, based on the correlation between local edge awareness and global graph awareness, allows us to train on modestly-scaled instances and generalize to much larger

TABLE V
PERFORMANCE COMPARISON OF SS2V-D3QN WITH BASELINE SOLVERS ON REAL-WORLD MCMP INSTANCES. IN EACH ITEM,
A TOTAL OBJECTIVE VALUE OF ALL OBTAINED SOLUTIONS FOR THE TEST INSTANCE SET IS REPORTED (LOWER IS BETTER)

Solvers	image-seg	knott-3d-150	knott-3d-300	knott-3d-450	karate	lesmis	football	SNEMI3D	CREMI-C	FIB25
CPLEX	-4227.514	-14786.006	-11915.077	-9428.235	-11.550	-3.115	-1.644	-55092.405	-51654.479	-44469.986
CGC	-4223.502	-14783.442	-11915.077	-9420.801	-11.373	-3.112	-1.634	-55092.405	-51654.479	-44465.205
FM (GAEC)	-4220.934	-14782.788	-11915.077	-9419.739	-11.399	-3.109	-1.635	-55092.405	-51654.479	-44456.909
FM (KLj)	-4220.934	-14782.788	-11915.077	-9419.739	-11.399	-3.109	-1.635	-55092.405	-51654.479	-44468.604
KLj	-4221.183	-14783.858	-11915.077	-9425.949	-11.486	-3.112	-1.641	-55092.405	-51654.479	-44469.986
BEST	-2373.008	-10634.907	-6025.394	-1437.649	-6.154	-2.625	-1.347	-53539.073	-49085.172	-42005.504
FIRST	-1750.333	-9232.432	-4463.568	167.467	-4.436	-2.492	-1.235	-52935.806	-48508.542	-41164.959
VOTE	-2621.728	-11989.519	-7070.237	-2939.681	-7.758	-2.901	-1.467	-53845.072	-49579.029	-42356.947
PIVOT	-700.717	-5926.857	-340.977	4902.810	-5.647	-2.352	-1.291	-51210.506	-47336.563	-39653.907
GAEC	-4219.695	-14780.568	-11915.077	-9419.504	-11.361	-3.107	-1.630	-55092.405	-51653.633	-44455.218
GF	-4184.872	-14453.954	-11687.152	-9281.934	-8.669	-2.648	-1.007	-54975.975	-51598.585	-44290.772
BEC	-4200.136	-14762.980	-11878.215	-9372.518	-11.383	-3.100	-1.631	-55080.276	-51651.405	-44452.167
Ours ($H = 1$)	<u>-4220.369</u>	<u>-14781.907</u>	-11915.077	<u>-9426.607</u>	<u>-11.459</u>	-3.105	-1.634	-55092.405	-51654.479	-44469.986
Ours ($H = 10$)	-4224.293	-14783.703	-11915.077	-9426.878	-11.509	-3.115	-1.641	-55092.405	-51654.479	-44469.986

For each instance set, the top ranking among all solvers except CPLEX is highlighted in **bold**, while the best one among the solvers in a monotone coarsening fashion is underlined.

ones from the same graph model during testing. To investigate this capability, we apply the learned policies collected before to these much larger ones (involving 80, 100, or 120 nodes) generated from the same graph model, as shown in Table III. Specifically, the applied ten policies include the best 3, 4, and 3 policies learned from instances with 20, 40, and 60 nodes, respectively. Although those learning-free solvers are out of generalization ability, we still provide their results for the convenience of evaluation. The overall results suggest that, for much larger instances from the same model, the SS2V-D3QN with polynomial complexity remains competitive even compared to reversible move-making algorithms such as FM and KLj, especially for the scale-free model BA. However, our method generalizes less well for the most random model ER, which is sensitive to instance scales.

E. Ablation Study for SS2V

We investigate the necessity of extracting bilevel features from both contracted and original graphs using the SS2V network, as well as explore the optimal numbers of iterations for the subnetworks. As shown in Table IV, the necessity is confirmed by comparing it with extracting single-level features. Furthermore, the results demonstrate that the SS2V-D3QN achieves peak performance when each subnetwork iterates twice, which aligns with the general consensus that overly deep graph neural networks may not be advantageous.

F. Sensitivity to H in Ensemble Inference

We assess the sensitivity of SS2V-D3QN performance to H in ensemble inference. Fig. 5 demonstrates that, as H increases from 1 to roughly 3, a notable improvement in solving performance is observed. Moreover, $H = 10$ is an appropriate choice for achieving state-of-the-art performance, while maintaining computational efficiency.

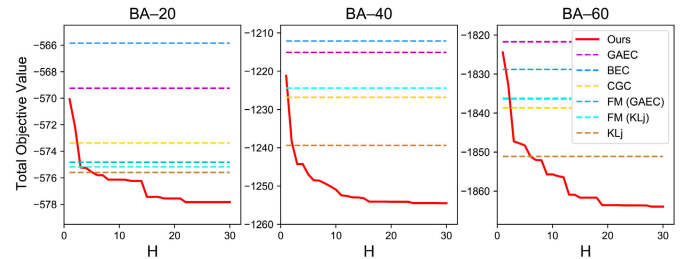


Fig. 5. Sensitivity of SS2V-D3QN performance to H on BA instances of three different scales. Results of other competitive solvers are also included for comparison.

G. On Real-World Instances

Numerous real-world graph partitioning tasks, including image/volume segmentation and modularity clustering, can be modeled as the MCMP. First, we compare the performance of the proposed SS2V-D3QN against baseline solvers on real-world MCMP instances from the OpenGM2 benchmark [80], consisting of seven instance sets (distributions) categorized into three types.

- 1) *image-seg* [81] consists of instances w.r.t. planar region adjacency graphs (RAGs) of image superpixels from the Berkeley segmentation dataset [82], which we have randomly cropped into smaller instances, each involving 50 nodes and about 120 edges.
- 2) *knott-3d-150/300/450* [13], [83] contain instances w.r.t. nonplanar RAGs arising from 3-D volumes acquired by serial block-face scanning electron microscopy (EM). Each cropped instance involves 50 nodes and about 220 edges.
- 3) *karate*, *lesmis*, and *football* from modularity clustering [84] contain instances on complete graphs derived from clustering problems on (social) interaction networks. Given the complexity imposed by complete graphs, we crop instances into 25 nodes and 300 edges each.

In addition, since superpixel aggregation in EM segmentation represents a significant real-world application of the multicut

problem, three other representative EM-related instance sets are included for comparison: SNEMI3D² [85], CREMI-C,³ and FIB25⁴ [86]. These sets are newly generated by cropping original instances into smaller ones, each involving 50 nodes and about 220 edges. As shown in Table V, experimental results demonstrate the superiority of our SS2V-D3QN over other combinatorial solvers, no matter which category of comparison is considered.

VII. SUMMARY AND DISCUSSION

In this article, we formulate solving the NP-hard multicut problem as a sequential decision process w.r.t. edge contraction, namely, EC-MDP, and propose a deep graph reinforcement learning method called SS2V-D3QN. The method can learn to infer feasible multicut solutions end-to-end toward optimization of the multicut objective in a data-driven manner. More essentially, by exploring the decision space adaptively, it can implicitly gain heuristic knowledge from topological patterns of similar instances and thereby generate more targeted heuristics overcoming the short-sightedness inherent in the conventional, learning-free, and hand-designed ones. Be aware that the bilevel subgraph neural network, SS2V, is customized for the dynamically edge-contracted graph environment by extracting features from both contracted and original graphs. To the best of the authors' knowledge, this work advances the field as the first to employ machine learning for solving the multicut problem end-to-end without extra postprocessing. Extensive experiments on synthetic and real-world MCMC instances demonstrate the superior effectiveness of the polynomial SS2V-D3QN compared with the existing combinatorial solvers and even the out-of-distribution generalization ability to some extent.

This work adds a primitive of deep graph reinforcement learning to the multicut problem community and opens new avenues for future research. Opportunities to improve the current work fall into six aspects.

- 1) Utilizing more advanced reinforcement learning algorithms to fit the optimal parameterized Q -function.
- 2) Incorporating the information from (Lagrangian) dual optimization for representing action edges.
- 3) Enabling reversible actions like move-making within the sequential decision framework.
- 4) Integrating the divide-and-conquer subgraph decomposition [15], [87], [88] and the proposed method for solving much larger multicut instances.
- 5) Extending the proposed method to apply to more graph partitioning problems, such as lifted multicut [22] and higher-order multicut [25].
- 6) Extending the proposed method to directed graphs.

ACKNOWLEDGMENT

The authors would like to thank the Transdisciplinary Platform of Functional Connectome and Brain-Inspired Intelligence, Huairou Science City, Beijing, China, for its technical support and provision of device resources.

²<http://brainiac2.snemi3d.org/SNEMI3D/>

³<https://cremi.org/>

⁴<https://www.janelia.org/tools-and-data-release>

REFERENCES

- [1] B. H. Korte, J. Vygen, B. Korte, and J. Vygen, *Combinatorial Optimization*, vol. 1. Berlin, Germany: Springer, 2011.
- [2] S. Chopra and M. R. Rao, "The partition problem," *Math. Program.*, vol. 59, no. 1, pp. 87–115, Mar. 1993.
- [3] N. Bansal, A. Blum, and S. Chawla, "Correlation clustering," *Mach. Learn.*, vol. 56, no. 1, pp. 89–113, Jul. 2004.
- [4] E. D. Demaine, D. Emanuel, A. Fiat, and N. Immerlica, "Correlation clustering in general weighted graphs," *Theor. Comput. Sci.*, vol. 361, nos. 2–3, pp. 172–187, Sep. 2006.
- [5] N. Cesa-Bianchi, C. Gentile, F. Vitale, and G. Zappella, "A correlation clustering approach to link classification in signed networks," in *Proc. 25th Annu. Conf. Learn. Theory*, 2012, pp. 34.1–34.20.
- [6] F. Chierichetti, N. Dalvi, and R. Kumar, "Correlation clustering in MapReduce," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2014, pp. 641–650.
- [7] K. Ahn, G. Cormode, S. Guha, A. McGregor, and A. Wirth, "Correlation clustering in data streams," in *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 2237–2246.
- [8] A. Ukkonen, "Crowdsourced correlation clustering with relative distance comparisons," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2017, pp. 1117–1122.
- [9] J. H. Kappes, M. Speth, B. Andres, G. Reinelt, and C. Schn, "Globally optimal image partitioning by multicuts," in *Proc. Int. Conf. Energy Min. Methods Comput. Vis. Pattern Recognit.*, 2011, pp. 31–44.
- [10] A. Alush and J. Goldberger, "Ensemble segmentation using efficient integer linear programming," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 10, pp. 1966–1977, Oct. 2012.
- [11] M. Keuper, B. Andres, and T. Brox, "Motion trajectory segmentation via minimum cost multicuts," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 3271–3279.
- [12] A. Kirillov, E. Levinkov, B. Andres, B. Savchynskyy, and C. Rother, "InstanceCut: From edges to instances with MultiCut," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 5008–5017.
- [13] B. Andres et al., "Globally optimal closed-surface segmentation for connectomics," in *Proc. Eur. Conf. Comput. Vis.*, 2012, pp. 778–791.
- [14] T. Beier et al., "Multicut brings automated neurite segmentation closer to human performance," *Nature Methods*, vol. 14, no. 2, pp. 101–102, Feb. 2017.
- [15] C. Pape, T. Beier, P. Li, V. Jain, D. D. Bock, and A. Kreshuk, "Solving large multicut problems for connectomics via domain decomposition," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops (ICCVW)*, Oct. 2017, pp. 1–10.
- [16] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
- [17] X. Zhang, H. Liu, Q. Li, and X.-M. Wu, "Attributed graph clustering via adaptive graph convolution," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 4327–4333.
- [18] D. Bo, X. Wang, C. Shi, M. Zhu, E. Lu, and P. Cui, "Structural deep clustering network," in *Proc. World Wide Web Conf.*, 2020, pp. 1400–1410.
- [19] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*. New York, NY, USA: Plenum Press, 1972, pp. 85–103.
- [20] M. Deza, M. Grötschel, and M. Laurent, "Clique-web facets for multicut polytopes," *Math. Operations Res.*, vol. 17, no. 4, pp. 981–1000, Nov. 1992.
- [21] M. M. Sørensen, "Facet-defining inequalities for the simple graph partitioning polytope," *Discrete Optim.*, vol. 4, no. 2, pp. 221–231, Jun. 2007.
- [22] M. Keuper, E. Levinkov, N. Bonneel, G. Lavoué, T. Brox, and B. Andres, "Efficient decomposition of image and mesh graphs by lifted multicuts," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1751–1759.
- [23] T. Kroeger, J. H. Kappes, T. Beier, U. Koethe, and F. A. Hamprecht, "Asymmetric cuts: Joint image labeling and partitioning," in *Proc. 36th DAGM Ger. Conf. Pattern Recognit.*, in Lecture Notes in Computer Science, vol. 8753, 2014, pp. 199–211.
- [24] N. E. Krasowski, T. Beier, G. W. Knott, U. Köthe, F. A. Hamprecht, and A. Kreshuk, "Neuron segmentation with high-level biological priors," *IEEE Trans. Med. Imag.*, vol. 37, no. 4, pp. 829–839, Apr. 2018.
- [25] S. Kim, C. D. Yoo, S. Nowozin, and P. Kohli, "Image segmentation using higher-order correlation clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 9, pp. 1761–1774, Sep. 2014.

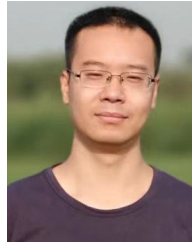
- [26] E. Levinkov, A. Kardoost, B. Andres, and M. Keuper, "Higher-order multicuts for geometric model fitting and motion segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 608–622, Jan. 2023.
- [27] V. Ng and C. Cardie, "Improving machine learning approaches to coreference resolution," in *Proc. 40th Annu. Meet. Assoc. Comput. Linguist.*, 2002, pp. 104–111.
- [28] W. M. Soon, H. T. Ng, and D. C. Y. Lim, "A machine learning approach to coreference resolution of noun phrases," *Comput. Linguistics*, vol. 27, no. 4, pp. 521–544, Dec. 2001.
- [29] M. Elsner and E. Charniak, "You talking to me? A corpus and algorithm for conversation disentanglement," in *Proc. 46th Annu. Meeting Assoc. Comput. Linguist. Hum. Lang. Technol.*, 2008, pp. 834–842.
- [30] N. Ailon, M. Charikar, and A. Newman, "Aggregating inconsistent information: Ranking and clustering," *J. ACM*, vol. 55, no. 5, pp. 23:1–23:27, Oct. 2008.
- [31] E. Levinkov, A. Kirillov, and B. Andres, "A comparative study of local search algorithms for correlation clustering," in *Proc. 39th DAGM Ger. Conf. Pattern Recognit.*, in Lecture Notes in Computer Science, vol. 10496, 2017, pp. 103–114.
- [32] A. Kardoost and M. Keuper, "Solving minimum cost lifted multicut problems by node agglomeration," in *Proc. 14th Asian Conf. Comput. Vis.*, 2018, pp. 74–89.
- [33] B. W. Kernighan and S. Lin, "An efficient heuristic procedure for partitioning graphs," *Bell Syst. Tech. J.*, vol. 49, no. 2, pp. 291–307, Feb. 1970.
- [34] A. Gionis, H. Mannila, and P. Tsaparas, "Clustering aggregation," *ACM Trans. Knowl. Discov. Data*, vol. 1, no. 1, pp. 1–30, Mar. 2007.
- [35] T. Beier, T. Kroeger, J. H. Kappes, U. Kothe, and F. A. Hamprecht, "Cut, glue & cut: A fast, approximate solver for multicut partitioning," *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 73–80, 2014.
- [36] T. Beier, F. A. Hamprecht, and J. H. Kappes, "Fusion moves for correlation clustering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3507–3516.
- [37] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA, USA: MIT Press, 2018.
- [38] C. Jia, F. Zhang, T. Xu, J.-C. Pang, Z. Zhang, and Y. Yu, "Model gradient: Unified model and policy learning in model-based reinforcement learning," *Frontiers Comput. Sci.*, vol. 18, no. 4, Aug. 2024, Art. no. 184339.
- [39] S. Nowozin and S. Jegelka, "Solution stability in linear programming relaxations: Graph partitioning and unsupervised learning," in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, Jun. 2009, pp. 769–776.
- [40] M. Grötschel and Y. Wakabayashi, "A cutting plane algorithm for a clustering problem," *Math. Program.*, vol. 45, pp. 59–96, Aug. 1989.
- [41] M. Grötschel and Y. Wakabayashi, "Facets of the clique partitioning polytope," *Math. Program.*, vol. 47, pp. 367–387, May 1990.
- [42] IBM ILOG CPLEX Optimization Studio. (2022). *User's Manual for CPLEX*. [Online]. Available: <https://www.ibm.com/docs/en/icos/22.1.1?topic=optimizers-users-manual-cplex>
- [43] M. Elsner and W. Schudy, "Bounding and comparing methods for correlation clustering beyond ILP," in *Proc. NAACL-HLT Workshops*, 2009, pp. 19–27.
- [44] P. Swoboda and B. Andres, "A message passing algorithm for the minimum cost multicut problem," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4990–4999.
- [45] J.-H. Lange, A. Karrenbauer, and B. Andres, "Partial optimality and fast lower bounds for weighted correlation clustering," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 2892–2901.
- [46] A. Abbas and P. Swoboda, "RAMA: A rapid multicut algorithm on GPU," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 8183–8192.
- [47] S. Jung and M. Keuper, "Learning to solve minimum cost multicuts efficiently using edge-weighted graph convolutional neural networks," in *Proc. Mach. Learn. Knowl. Discov. Databases (ECML PKDD)*, in Lecture Notes in Computer Science, vol. 13714, 2022, pp. 485–501.
- [48] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2692–2700.
- [49] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," in *Proc. 5th Int. Conf. Learn. Represent. Workshop*, 2017, pp. 1–5.
- [50] H. Dai, E. Khalil, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6351–6361.
- [51] T. Barrett, W. Clements, J. Foerster, and A. Lvovsky, "Exploratory combinatorial optimization with reinforcement learning," in *Proc. 34th AAAI Conf. Artif. Intell.*, 2020, pp. 3243–3250.
- [52] I. Drori et al., "Learning to solve combinatorial optimization problems on real-world graphs in linear time," in *Proc. 19th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2020, pp. 19–24.
- [53] C. Liu, Z. Jiang, R. Wang, J. Yan, L. Huang, and P. Lu, "Revocable deep reinforcement learning with affinity regularization for outlier-robust graph matching," in *Proc. 11th Int. Conf. Learn. Represent.*, 2023, pp. 1–25.
- [54] C.-L. Liu and T.-H. Huang, "Dynamic job-shop scheduling problems using graph neural network and deep reinforcement learning," *IEEE Trans. Syst. Man Cybern., Syst.*, vol. 53, no. 11, pp. 6836–6848, Nov. 2023.
- [55] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *Eur. J. Oper. Res.*, vol. 290, no. 2, pp. 405–421, Apr. 2021.
- [56] Y. Peng, B. Choi, and J. Xu, "Graph learning for combinatorial optimization: A survey of state-of-the-art," *Data Sci. Eng.*, vol. 6, no. 2, pp. 119–141, Apr. 2021.
- [57] N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev, "Reinforcement learning for combinatorial optimization: A survey," *Comput. Oper. Res.*, vol. 134, Oct. 2021, Art. no. 105400.
- [58] J. You, B. Liu, Z. Ying, V. Pande, and J. Leskovec, "Graph convolutional policy network for goal-directed molecular graph generation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 6412–6422.
- [59] H. Dai et al., "Adversarial attack on graph structured data," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1115–1124.
- [60] Y. Sun, S. Wang, X. Tang, T.-Y. Hsieh, and V. Honavar, "Adversarial attacks on graph neural networks via node injections: A hierarchical reinforcement learning approach," in *Proc. World Wide Web Conf.*, 2020, pp. 673–683.
- [61] V.-A. Darvari, S. Hailes, and M. Musolesi, "Goal-directed graph construction using reinforcement learning," *Proc. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 477, no. 2254, Oct. 2021, Art. no. 20210168.
- [62] H. Dai, Y. Li, C. Wang, R. Singh, P.-S. Huang, and P. Kohli, "Learning transferable graph exploration," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 2514–2525.
- [63] C. Fan, L. Zeng, Y. Sun, and Y.-Y. Liu, "Finding key players in complex networks through deep reinforcement learning," *Nature Mach. Intell.*, vol. 2, no. 6, pp. 317–324, May 2020.
- [64] Y. Zheng, Y. Lin, L. Zhao, T. Wu, D. Jin, and Y. Li, "Spatial planning of urban communities via deep reinforcement learning," *Nature Comput. Sci.*, vol. 3, no. 9, pp. 748–762, Sep. 2023.
- [65] S. Munikoti, D. Agarwal, L. Das, M. Halappanavar, and B. Natarajan, "Challenges and opportunities in deep reinforcement learning with graph neural networks: A comprehensive review of algorithms and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jun. 26, 2023, doi: 10.1109/TNNLS.2023.3283523.
- [66] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, pp. 279–292, May 1992.
- [67] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [68] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
- [69] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th Conf. Artif. Intell. (AAAI)*, Mar. 2016, pp. 2094–2100.
- [70] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [71] C. Huang, M. Li, F. Cao, H. Fujita, Z. Li, and X. Wu, "Are graph convolutional networks with random weights feasible?" *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2751–2768, Mar. 2023.
- [72] M. Li, L. Zhang, L. Cui, L. Bai, Z. Li, and X. Wu, "BLoG: Bootstrapped graph representation learning with local and global regularization for recommendation," *Pattern Recognit.*, vol. 144, Dec. 2023, Art. no. 109874.
- [73] J. Luo, M. He, W. Pan, and Z. Ming, "BGNN: behavior-aware graph neural network for heterogeneous session-based recommendation," *Frontiers Comput. Sci.*, vol. 17, no. 5, Oct. 2023, Art. no. 175336.
- [74] H. Dai, B. Dai, and L. Song, "Discriminative embeddings of latent variable models for structured data," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 2702–2711.

- [75] I. Loshchilov and F. Hutter. (2018). *Fixing Weight Decay Regularization in Adam*. [Online]. Available: <https://openreview.net/forum?id=rk6qdGgCZ>
- [76] L. N. Smith, "Cyclical learning rates for training neural networks," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Mar. 2017, pp. 464–472.
- [77] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999.
- [78] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, no. 6684, pp. 440–442, Jun. 1998.
- [79] P. Erdős and A. Rényi, "On the evolution of random graphs," *Publ. Math. Inst. Hung. Acad. Sci.*, vol. 5, pp. 17–60, 1960.
- [80] J. H. Kappes et al., "A comparative study of modern inference techniques for structured discrete energy minimization problems," *Int. J. Comput. Vis.*, vol. 115, no. 2, pp. 155–184, Mar. 2015.
- [81] B. Andres, J. H. Kappes, T. Beier, U. Köthe, and F. A. Hamprecht, "Probabilistic image segmentation with closedness constraints," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2011, pp. 2611–2618.
- [82] D. Martin, C. Fowlkes, D. Tal, and J. Malik, "A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics," in *Proc. IEEE Int. Conf. Comput. Vis.*, vol. 2, 2001, pp. 416–423.
- [83] B. Andres et al., "3D segmentation of SBFSEM images of neuropil by a graphical model over supervoxel boundaries," *Med. Image Anal.*, vol. 16, no. 4, pp. 796–805, May 2012.
- [84] U. Brandes et al., "On modularity clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 20, no. 2, pp. 172–188, Feb. 2008.
- [85] I. Arganda-Carreras et al., "Crowdsourcing the creation of image segmentation algorithms for connectomics," *Frontiers Neuroanatomy*, vol. 9, p. 142, Nov. 2015.
- [86] S.-Y. Takemura et al., "Synaptic circuits and their variations within different columns in the visual system of drosophila," *Proc. Nat. Acad. Sci. USA*, vol. 112, no. 44, pp. 13711–13716, Nov. 2015.
- [87] X. Pan, D. Papailiopoulos, S. Oymak, B. Recht, K. Ramchandran, and M. I. Jordan, "Parallel correlation clustering on big graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 82–90.
- [88] K. Ho, A. Chatzimichailidis, M. Keuper, and J. Keuper, "MSM: Multi-stage multicuts for scalable image clustering," in *Proc. ISC High Perform. High Perform. Comput. Workshops*, in Lecture Notes in Computer Science, vol. 12761, 2021, pp. 267–284.



Yanchao Zhang received the B.E. degree from Northeastern University, Shenyang, China, in 2021. He is currently pursuing the Ph.D. degree with the University of Chinese Academy of Sciences, Beijing, China.

His research interests include connectomics and computer vision.



Shaofeng Zeng received the Ph.D. degree in computer science and technology from Beijing University of Technology, Beijing, China, in 2018.

His research interests include basic methodologies and applications in computer vision.



Jingbin Yuan received the M.E. degree from Harbin University of Science and Technology, Harbin, China, in 2020. He is currently pursuing the Ph.D. degree with the University of Chinese Academy of Sciences, Beijing, China.

His research interests include connectomics and large-scale data management.



Jiazheng Liu received the Ph.D. degree in pattern recognition and intelligent systems from the University of Chinese Academy of Sciences, Beijing, China, in 2024.

His research interests include deep learning and image processing.



Zhenchen Li received the B.S. degree in mathematics from Xidian University, Xi'an, China, in 2019. He is currently pursuing the Ph.D. degree with the University of Chinese Academy of Sciences, Beijing, China.

His research interests include combinatorial optimization, biomedical image analysis, and machine learning.



Zhiyong Liu (Senior Member, IEEE) is currently a Professor with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing, China. His research interests include machine learning, pattern recognition, computer vision, and bioinformatics.



Xu Yang (Senior Member, IEEE) is currently an Associate Professor with the State Key Laboratory of Multimodal Artificial Intelligence Systems, Institute of Automation, Chinese Academy of Sciences, Beijing, China. His research interests include computer vision, graph algorithms, and robotics.



Hua Han received the Ph.D. degree from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2004.

He is currently a Professor with the Key Laboratory of Brain Cognition and Brain-inspired Intelligence Technology, Institute of Automation, Chinese Academy of Sciences. His research interests include pattern recognition, image processing, and computational neuroscience.